

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – Pseudocode Writing Task**

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
Student Name:	Santhosh G	Reg. No.:	192521035
Section / Batch:		Date:	24-8-26

Instructions to Students

- Answer BOTH questions with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions	Marks
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1 – Q2	Q1 – 50 Q2 – 50
GRAND TOTAL:			100 Marks

SECTION A – PSEUDOCODE WRITING TASK**Q1. Approximation Vertex Cover Problem**

Develop a pseudocode to compute a 2-approximation solution for Vertex Cover. Clearly identify inputs (graph) and output (vertex set). Design an algorithm selecting edges and covering vertices. Use loops effectively. Ensure correctness of approximation logic. Present structured pseudocode. Handle all cases.

Q2. Combination Generation with Sum Constraint

Design a pseudocode using Backtracking to generate combinations of size k whose sum equals a target value. Clearly identify inputs (set, k , target) and output (valid combinations). Develop an algorithm that selects elements incrementally. Use loops and recursive calls for exploration. Apply pruning to discard combinations exceeding target sum. Ensure clarity and structure. Handle all valid combinations.

Q.1. Approximation Vertex Cover Problem:

Develop a pseudocode to compute a 2-approximation solution for vertex cover. Clearly identify inputs (graph) and output (vertex set). Design an algorithm selecting edges and covering vertices. Use loops effectively. Ensure correctness of approximation logic. Present structured pseudocode. Handle all cases.

Input: An undirected graph $G = (V, E)$, where V is set of vertices and E is the set of edges.

Output: A vertex set $C \subseteq V$ such that every edge has at least one endpoint in C .

Pseudocode:

ALGORITHM ApproxVertexCover($G = (V, E)$)

INPUT:

$G = (V, E)$, an undirected graph

OUTPUT:

C , a vertex cover of G

$C \leftarrow \emptyset$

$F \leftarrow E$

While $F \neq \emptyset$ DO

 Select any edge (u, v) from F

$C \leftarrow C \cup \{u, v\}$

 for each edge (x, y) in F DO

 if $x = u$ OR $x = v$ OR $y = u$ OR $y = v$ THEN

 Remove (x, y) from F

 END IF

 END FOR

END WHILE

RETURN C

Step by step logic:

- * Initially, no vertices are selected: $C = \emptyset$.
- * Pick any remaining uncovered edge (u, v) .
- * Add Both u and v to C .
- * Since u and v are now in the Cover, every edge incident to either vertex is covered.
- * Remove these covered edges from consideration.
- * Repeat until no edges remain.
- * At termination, every original edge has been covered, so C is a valid vertex cover.

Why is it a 2-approximation:

$$OPT \geq |M|$$

$$|C| = 2|M|$$

$$|C| = 2|M| \leq 2OPT$$

$$\boxed{|C| \leq 2OPT}$$

Complexity:

With an adjacency list representation, the implementation can be made $O(|V| + |E|)$ using appropriate data structures for maintaining uncovered edges and incident edges.

$$\boxed{O(|E|^2)}$$

in the worst case

Space Complexity:

$$O(|V| + |E|)$$

for storing the graph, edge set, and cover.

Q2. Combination Generation with Sum Constraint

Design a pseudo code using backtracking to generate combinations of size k whose sum equals a target value. Clearly identify inputs (set, k , target) and output (Valid combinations). Develop an algorithm that selects elements incrementally. Use loops and recursive calls for exploration. Apply pruning to discard combinations exceeding target sum. Ensure clarity and structure. Handle all valid combinations.

Input:

- * A Set/array A containing n elements
- * An integer k , the required combination size.
- * A target value T .

Output: All combinations of exactly k elements whose sum is exactly T .

State - Space representation:

A state can be represented as:

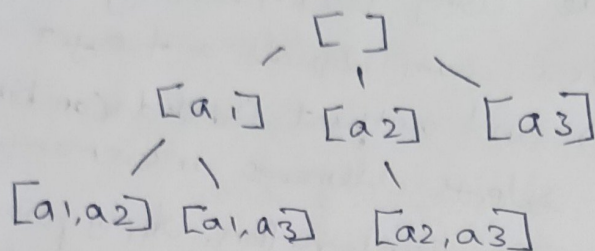
$(start, current, sum)$

Where:

- * $start$ = index from which the next element can be selected.
- * $current$ = elements currently selected.
- * sum = sum of elements in current.

A state-space tree represents all possible combinations.

for example:



A node is a solution when:

$|current| = k$ and $sum = T$.

Pseudo code:

ALGORITHM GenerateCombinations (k, T)

INPUT:

$A[0 \dots n-1]$

k

T

OUTPUT:

All k -element combinations whose sum is T .

result $\leftarrow$ empty list

current $\leftarrow$ empty list

IF $k < 0$ OR $k > n$ THEN

RETURN result

ENDIF

BACKTRACK (0, 0, current)

RETURN result

PROCEDURE BACKTRACK (start, currentSum, current)

IF $|current| = k$ THEN

IF currentSum = T then

Add a copy of current to result.

END IF

RETURN

END IF

remaining $\leftarrow k - |current|$

IF $n - start < remaining$ then

RETURN

END IF

FOR $i \leftarrow start$ TO $n - remaining$ DO

IF $current\ sum + A[i] > T$ Then

CONTINUE

END IF

ADD $A[i]$ to $current$

BACKTRACK($i+1$, $current\ sum + A[i]$, $current$)

Remove the last element from $current$

END FOR.

Recursion steps:

At each recursive call:

- * Determine how many elements are still required
- * Try every possible next element
- * Add the selected element to $current$.
- * Update $current\ sum$.
- * Recursively search for the remaining elements.
- * After returning, remove the selected element.
- * Try the next candidate

Example:

$A = \{2, 3, 5, 7, 8\}$, $k = 2$, $T = 10$.

Therefore the output is:

$$\{2, 8\}, \{3, 7\}$$

Complexity analysis:

There can be

$$\binom{n}{k}$$

* Standard worst-case bound is:

$$O\left(\left(\binom{n}{k}\right) \cdot k\right)$$

Including the output, Space can be:

$$O\left(k \binom{n}{k}\right)$$

in the worst case.

Sorting if required, takes:

$$O(n \log n)$$

and is performed once before the backtracking search.

Code of Conduct

I SANTHOSH G - 192521035 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Santhosh

Marks Evaluation:

Question No.	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient